Reviewer Pack — Minimal Rank of Affine Representations in Representation The…

Entry f2de4e8b6500 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 02:00:03 UTC

Minimal Rank of Affine Representations in Representation Theory × Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: `f2de4e8b6500` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 02:00:03 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory: Affine Representations **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

> ['For any affine representation V of a finite group G, the minimal rank of V is at least n^(1/2), where n is the number of variables required by a monotone circuit computing the k-CLIQUE function.', 'Equivalently, for every instance I of k-CLIQUE, there exists an affine representation of the incidence algebra of I with rank ≥ (number of vertices in I)^(1/2).', 'The rank of the affine representation is defined as the number of basis elements required to generate V.']

Rationale (proposer's reasoning):

> ['Affine representations have been used in other areas of mathematics to describe complex structures, and their ranks have been related to other computational problems. Investigating the relationship between the rank of an affine representation and the size of monotone circuits could expose new structural insights into k-CLIQUE complexity.', 'The conjecture leverages the tools from representation theory to address a fundamental problem in complexity theory, potentially leading to new algorithms or understanding of circuit lower bounds.']

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `dd0be282599b096f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of randomly generated k-CLIQUE instances with n ≤ 40 vertices have an affine representation with rank ≥ n^(1/2). The conjecture is falsified if any instance has an affine representation with rank < n^(1/2).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"affine representation" AND "monotone circuit lower bounds" AND k-CLIQUE` - `"minimal rank" INTEGRAL PARTIAL ORDERING "affine representation" AND complexity theory` - `"incidence algebra" AND k-CLIQUE AND rank >= (number of vertices)^0.5`

Top relevant hits considered: - [s2:hep-th/9808155] Extended Iterative Scheme for QCD: the Four-Gluon Vertex

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Find a non-zero pivot below
            for j in range(i + 1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                raise ValueError("Matrix is singular")
        factor = Fraction(1, A[i][i])
        for k in range(n):
            A[i][k] *= factor
        for j in range(i + 1, n):
            factor = A[j][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def rank_of_matrix(A):
    n = len(A)
    rank = 0
```

```
    for i in range(n):
        if all(A[i][j] == Fraction(0) for j in range(n)):
            continue
        rank += 1
        factor = Fraction(1, A[i][i])
        for k in range(n):
            A[i][k] *= factor
        for j in range(i + 1, n):
            factor = A[j][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return rank

def incidence_algebra(I):
    n = len(I)
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if I[i][j]:
                A[i][j] = Fraction(1, math.factorial(j - i - 1))
                A[j][i] = Fraction(-1, math.factorial(j - i - 1))
    return A

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    I = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        I[i][i] = 0
    G = incidence_algebra(I)
    rank = rank_of_matrix(G)
    metric_value = rank
    instances_tested = 1
    conjecture_holds = rank >= n ** Fraction(1, 2)
    counterexample = "" if conjecture_holds else "n={}".format(n)
    return {
        "metric_name": "affine_representation_rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 307))  # Default to first 30 primes if
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print("TRIAL: {}".format(trial_result))
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
```

```
    if all(result["conjecture_holds"] for result in results):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value,
    elif support_fraction >= 0.8:
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value,
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print("RESULT: FALSIFIED counterexample='n={}' first_failing_seed={}".format(first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d57b1439.py", line 89, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d57b1439.py", line 71, in run_trial
    rank = rank_of_matrix(G)
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d57b1439.py", line 45, in rank_of_matri
    factor = Fraction(1, A[i][i])
             ^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to a ZeroDivisionError, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the ZeroDivisionError in the code to allow for the completion of the test.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13337
2	propose	ollama_remote	glm4:latest	0	0	13625
3	preregistration	ollama_remote	glm4:latest	0	0	9130
4	novelty	ollama_remote	glm4:latest	0	0	8772

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	9058
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16047
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9531
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12572
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10636
10	judge	ollama_remote	glm4:latest	0	0	15469

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118177 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f2de4e8b6500.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f2de4e8b6500.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f2de4e8b6500.tar.gz` (if generated)